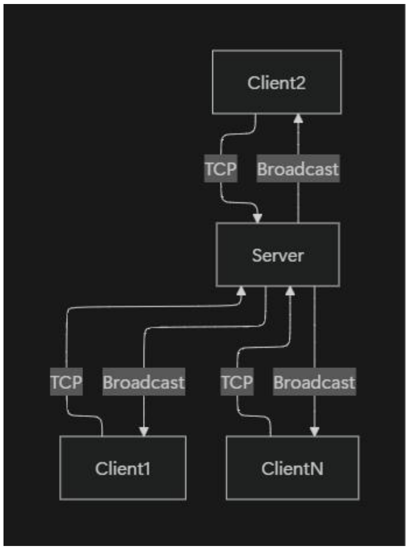
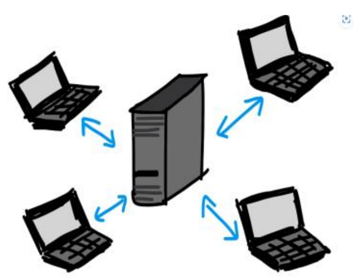
Technical Documentation (Advanced Version)

1. System Architecture Design.

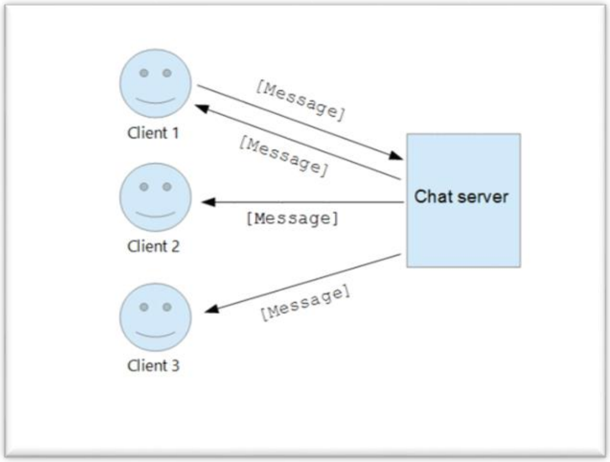


The system is implemented using a client-server architecture based on TCP sockets. The server acts as a central coordinator for managing multiple client connections, handling message distribution, and maintaining chat room structures. Each client establishes a persistent connection to the server and communicates through socket channels.



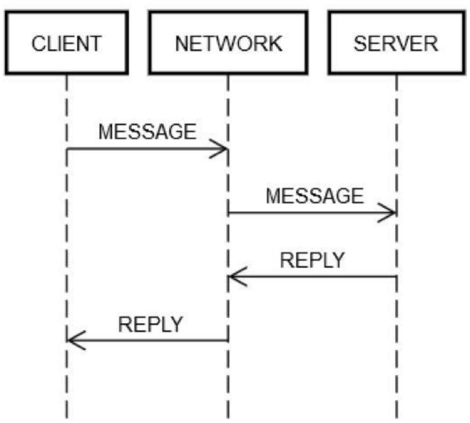
Client-Server Network Model

The architecture ensures connection for multiple users and supports concurrent communication through multithreading. The server maintains lists of active clients, associated nicknames, and room membership mappings.

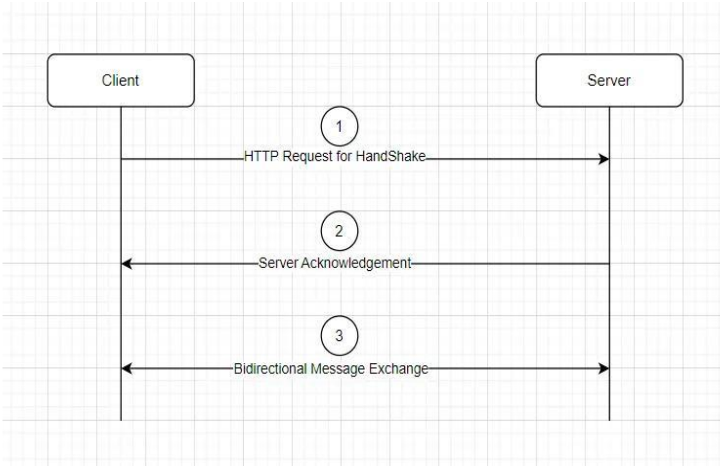


2. Detailed Protocol Specification

The communication protocol is a custom application-layer protocol built on top of TCP. Messages are transmitted as ASCII-encoded strings. The protocol defines specific control messages and data messages:



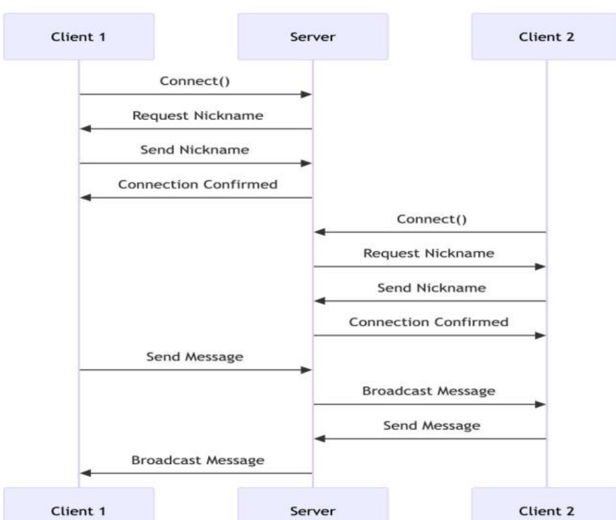
- NICK: Server requests a client nickname after connection establishment.
- DUPLICATE: Server rejects duplicate nicknames to maintain uniqueness.
- CHAT MESSAGE: Standard messages broadcast to users within a room.



All messages follow a simple structure without headers, relying on interpretation of content.

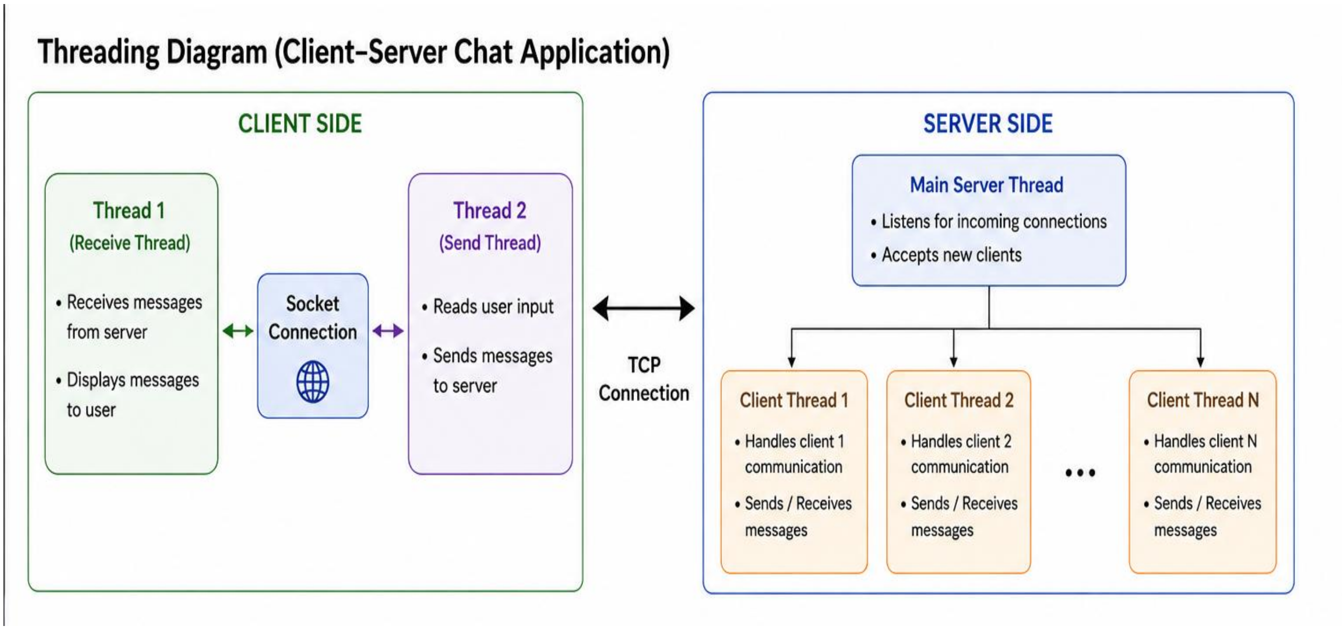
3. Network Communication Flow

Step 1: Client initiates TCP connection to the server.
Step 2: Server accepts connections and sends a NICK request.
Step 3: Client responds with a nickname.
Step 4: Server validates uniqueness and either accepts or rejects.
Step 5: Client is assigned to a default chat room.
Step 6: Messages sent by the client are broadcast to other users in the same room.
Step 7: Server manages disconnection and resource cleanup.



4. Code Structure and Design

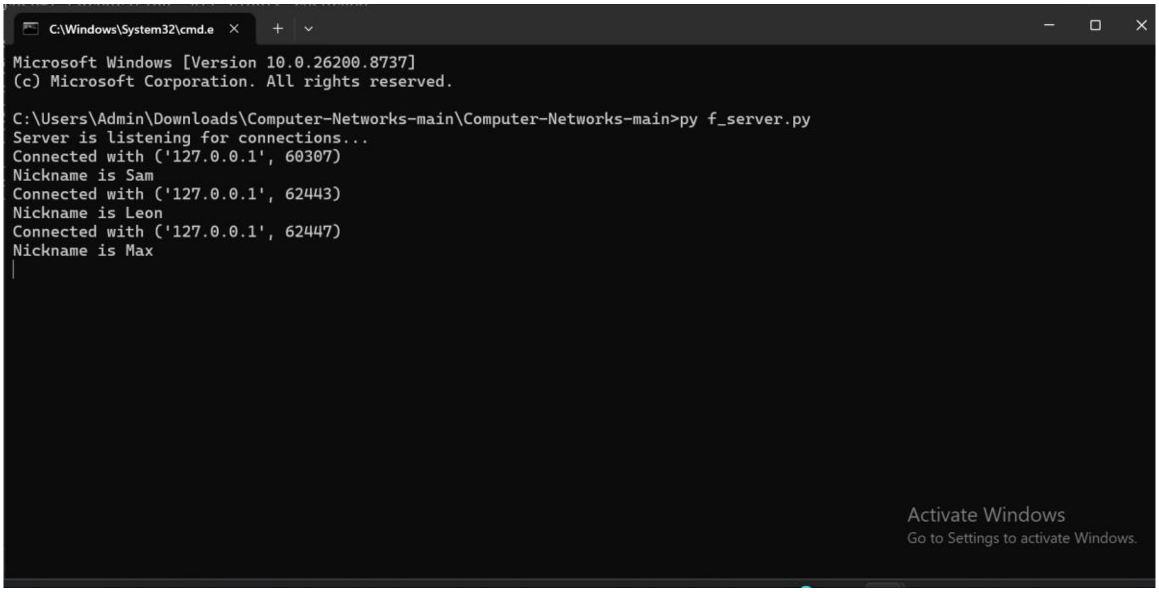
The client uses two threads: one for receiving messages and one for sending input, ensuring non-blocking communication. The server uses threading to handle multiple clients concurrently.



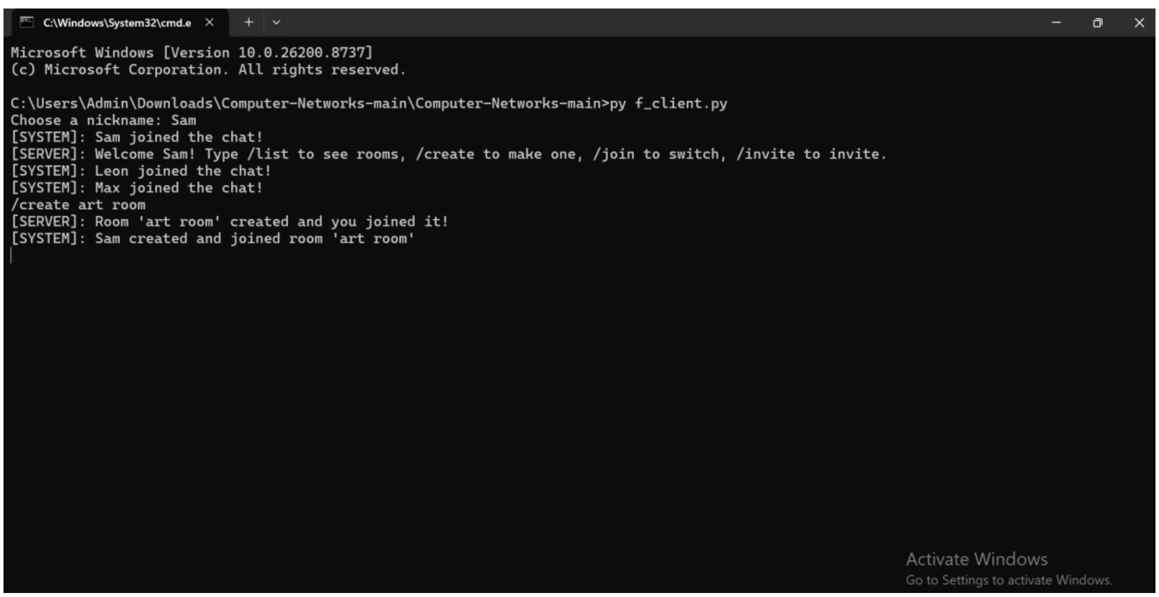
Key components:
- Socket management for connection handling
- Threading for concurrency
- Lists and dictionaries for state management
- Error handling using try-except blocks

5. Installation and Usage Guide

1. Modify the IP address in both server and client scripts.
2. Run the server script first.



3. Start multiple instances of the client script.
4. Enter unique nicknames when prompted.



5. Begin sending messages through the terminal interface.

6.Create various rooms for direct messaging.

7. Inviting friends to rooms.

8.Joining invited rooms to communicate.

C:\Windows\System32\cmd.e

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\Downloads\Computer-Networks-main\Computer-Network
s-main>py f_client.py
Choose your nickname: Sam
[SYSTEM]: Sam joined the chat!
[SERVER]: Welcome Sam! Type /list to see rooms, /create to make
one, /join to switch, /invite to invite.
[SYSTEM]: Leon joined the chat!
[SYSTEM]: Max joined the chat!
/create ball room
[SERVER]: Room 'ball room' created and you joined it!
[SYSTEM]: Sam created and joined room 'ball room'
/invite Leon
[SERVER]: Invite sent to Leon.
[SYSTEM]: Leon joined the room
/invite Max
[SERVER]: Invite sent to Max.
[SYSTEM]: Max joined the room
```